MediSafe Nepal

Dataset Expansion Guide: Adding New Nepali Drugs to the KnowDDI Graph

This guide explains, in full detail, how to expand the KnowDDI drug-drug interaction (DDI) dataset with drugs that are sold in Nepal but are **not** among the original 1,710 benchmark drugs. It is the core engineering procedure behind MediSafe Nepal, because the product's main limitation is dataset coverage: the model can only predict interactions for drugs that exist as nodes in the graph. Every new drug must be inserted into **three data files** with correct integer IDs, valid edges, and (where possible) biological context, after which the model is retrained.

Note: All operations are **additive**. You never renumber or delete existing IDs. New nodes take IDs above the current maximum. Back up every file before editing. This keeps the original 1,710-drug graph intact.

1. The Data Model You Are Editing

The model's input is a single **combined network** = the DDI graph (drug-drug interactions) merged with the Hetionet knowledge graph (drugs, genes, proteins, pathways, diseases). Everything is stored as integer IDs in one shared space:

ID range	What it is	Defined in
0 - 1709	Drug nodes (the original benchmark)	node2id.json
1710 - 34123	Knowledge-graph entities (genes, pathways, etc.)	BKG_entity2Id.json
34124 +	Free space for NEW drugs you add	(you assign these)

The three files you edit per new drug:

File	Purpose	Line format
node2id.json	Registers the drug as a node	`"DBxxxxx": <new_id>`
train.txt	DDI edges (drug-drug interactions)	`head_id tail_id type_id`
BKG_file.txt	Biological edges (drug-gene etc.)	`drug_id entity_id rel_id`

Supporting files you READ (never edit) to translate names and IDs:

- **id2rel.txt** – the 86 DDI interaction templates, each with #Drug1/#Drug2 slots and a Subject column that encodes direction.
- **relation_type_drug.json** – knowledge-graph relation names to integers (e.g. CbG = Compound-binds-Gene = 6).
- **hetionet-v1.0-nodes.tsv** – maps Gene::<entrez> to a human gene symbol (the bridge for biological edges).
- **drugbank.xml** – the full DrugBank export; the source of names, synonyms, interactions and gene targets.

2. End-to-End Pipeline (Overview)

For any batch of new Nepali drug names, the procedure is:

1. **Resolve** each name to a DrugBank ID (handle synonyms and salts).
2. **Classify**: already in 1,710 (skip), genuinely new (append), or out-of-scope (flag).
3. **Assign** new integer IDs (34124+) and write them to node2id.json.

4. **Extract DDI edges** from the XML, map each description to one of 86 types, keep only edges whose partner is already a node, append to train.txt.
5. **Extract biological edges** (drug to gene) via the symbol bridge, append to BKG_file.txt.
6. **Clear caches** and **retrain**.

3. Step 1 - Resolve Drug Names to DrugBank IDs

A drug name in Nepal may not match DrugBank's primary name. There are three match levels, tried in order:

- **Exact** – the name is DrugBank's primary <name> (case-insensitive).
- **Synonym** – the name appears in the drug's <synonyms> list (e.g. colecalciferol to Cholecalciferol; amidotrizoate to Diatrizoate).
- **Manual** – spelling errors, salt forms, or international names with no synonym entry (e.g. 'fomipezole' to fomepizole). These need a hand-built alias table.

```
import xml.etree.ElementTree as ET
NS='{http://www.drugbank.ca}'
name2db = {}
ctx = ET.iterparse('drugbank.xml', events=('end',))
for ev, el in ctx:
if el.tag == NS+'drug':
pid = None
for d in el.findall(NS+'drugbank-id'):
if d.get('primary')=='true': pid = d.text; break
nm = el.find(NS+'name')
if pid and nm is not None and nm.text:
name2db[nm.text.strip().lower()] = pid # exact
syn = el.find(NS+'synonyms')
if syn is not None:
for s in syn.findall(NS+'synonym'):
if s.text:
name2db.setdefault(s.text.strip().lower(), pid) # synonym
el.clear() # CRITICAL: free memory on the 700 MB file
```

Note: Always call **el.clear()** after each drug. The XML is ~700 MB; without clearing, the parser holds everything in RAM and the process is killed. Use iterparse, never ET.parse().

4. Step 2 - Classify Each Drug

Bucket	Test	Action
Already in 1,710	resolved DBID is a key in node2id.json	Skip - just fix your name->ID map
Out-of-scope	0 interactions in the XML	Flag 'no significant interaction'
Genuinely new	resolved, not in 1,710, has interactions	APPEND (steps 3-5)

Note: Out-of-scope drugs (topical antiseptics, contrast agents, vitamins, minerals, anaesthetic gases) genuinely have no meaningful drug-drug interactions. Flagging them is medically correct, not a failure. Do not force them into the model.

5. Step 3 - Assign IDs and Write node2id.json

```
import json
node2id = json.load(open('raw_data/Drugbank/node2id.json'))
bkg_ent = json.load(open('raw_data/Drugbank/BKG_entity2Id.json'))
next_id = max(max(node2id.values()), max(bkg_ent.values())) + 1 # 34124 on first run

new_ids = {}
for dbid in genuinely_new_dbids: # list from Step 2
new_ids[dbid] = next_id
next_id += 1
node2id.update(new_ids)
json.dump(node2id, open('raw_data/Drugbank/node2id.json','w'))
```

Caution: Compute next_id from the max of BOTH node2id and BKG_entity2Id. The next free number after the drugs (1710) already belongs to a gene. Starting at 34124 (current global max + 1) avoids overwriting a gene node.

6. Step 4 - DDI Edges into train.txt

6.1 Mapping description text to one of 86 types

Each <drug-interaction> in the XML has a partner DrugBank ID and a free-text description such as "The risk of bleeding can be increased when Apixaban is combined with Heparin." The 86 templates in id2rel.txt use #Drug1/#Drug2 placeholders. To map text to a type, replace the two real drug names with the placeholders and match the normalized sentence to a template. Direction (which drug is #Drug1) is recovered from which slot each name fills.

```
import csv, re
def norm(s):
return re.sub(r'\s+',' ', s.lower().strip()).rstrip('.')

templates = {} # normalized template -> type_id
with open('raw_data/Drugbank/id2rel.txt') as f:
r = csv.reader(f); next(r)
for row in r:
if len(row) >= 4:
templates[norm(row[1])] = int(row[0])

def map_desc(desc, self_name, partner_name):
# try both slot assignments; whichever matches a template wins
for self_slot, partner_slot, head_is_self in [('#drug2','#drug1',True),
('#drug1','#drug2',False)]:
x = desc
x = re.sub(re.escape(self_name), self_slot, x, flags=re.I)
x = re.sub(re.escape(partner_name), partner_slot, x, flags=re.I)
if norm(x) in templates:
return templates[norm(x)], head_is_self
return None # unmatched -> log, do NOT guess
```

Writing the edge, respecting direction and the partner-must-exist rule:

```
all_ids = dict(node2id) # includes the new drugs
train_lines, dropped, unmapped = [], 0, []
for dbid in genuinely_new_dbids:
h = node2id[dbid]
for partner_db, partner_name, desc in interactions[dbid]:
if partner_db not in all_ids: # partner not a node -> skip
dropped += 1; continue
t = all_ids[partner_db]
m = map_desc(desc, self_name[dbid], partner_name)
if m is None:
unmapped.append((dbid, partner_db, desc)); continue
type_id, head_is_self = m
a, b = (h, t) if head_is_self else (t, h)
train_lines.append(f"{a} {b} {type_id}")
with open('data/drugbank/train.txt','a') as f:
for ln in train_lines: f.write(ln+'\n')
```

Caution: Never invent a type for an unmatched description. In a clinical safety tool, a wrong interaction type produces wrong advice. Log unmatched descriptions to a CSV and review them, or leave that edge out.

6.2 Obstacles and fixes (DDI edges)

Obstacle	Why it happens	Fix
0 edges matched	real drug names still in the text; template only has #Drug1/#Drug2	Replace both drug names before matching (6.1)
Many partners dropped	partner drug is outside the 1,710	Expected. The node is still connected via other partners / KG. Add pa
Direction looks wrong	head/tail swapped vs the description	Use head_is_self from the matched slot, not a fixed order

A few % unmatched	DrugBank phrasing variant not in id2rel.txt	Log and skip; optionally add a manual phrase->type rule

7. Step 5 - Biological Edges into BKG_file.txt

Biological context is what makes KnowDDI strong: it lets the model reason about a drug through the genes and proteins it acts on, and connect it by similarity to other drugs. A new drug with DDI edges but no biological edges still works, but its predictions are weaker (the 'sparse drug' case). Add biological edges wherever the drug has known targets.

7.1 The gene-ID bridge (the key obstacle)

DrugBank's XML lists a drug's targets using gene **symbols** (e.g. SERPINC1) and UniProt codes - but **not** Entrez numbers. The knowledge graph names genes as Gene::<Entrez> (e.g. Gene::462). A direct match therefore fails. The bridge is the Hetionet node file, which maps Gene::<Entrez> to its symbol:

```
Gene::462 SERPINC1 Gene <-- from hetionet-v1.0-nodes.tsv
```

So the chain is: **gene symbol (XML)** → **Gene::<entrez> (hetionet tsv)** → **integer ID (BKG_entity2Id.json)**.

```
import json
# symbol -> Gene::entrez
symbol2gene = {}
with open('raw_data/hetionet/hetionet-v1.0-nodes.tsv') as f:
next(f)
for line in f:
p = line.rstrip().split('\t')
if len(p) >= 3 and p[2] == 'Gene':
symbol2gene[p[1].upper()] = p[0] # SERPINC1 -> Gene::462

bkg_ent = json.load(open('raw_data/Drugbank/BKG_entity2Id.json'))
CbG = 6 # Compound-binds-Gene, from relation_type_drug.json

bkg_lines, dropped_gene = [], 0
for dbid in genuinely_new_dbids:
h = node2id[dbid]
for symbol in target_symbols[dbid]: # from XML <targets> etc.
g = symbol2gene.get(symbol.upper())
if g is None or g not in bkg_ent:
dropped_gene += 1; continue
bkg_lines.append(f"{h} {bkg_ent[g]} {CbG}")
with open('data/drugbank/BKG_file.txt','a') as f:
for ln in bkg_lines: f.write(ln+'\n')
```

Reading the gene symbols from the XML (targets, enzymes, transporters, carriers):

```
target_symbols = {}
# inside the iterparse loop, for each wanted drug:
syms = []
for block in ['targets','enzymes','transporters','carriers']:
b = el.find(NS+block)
if b is None: continue
for t in b:
poly = t.find(NS+'polypeptide')
if poly is None: continue
sym = None
for xr in poly.iter(NS+'external-identifier'):
res = xr.find(NS+'resource'); idv = xr.find(NS+'identifier')
if res is not None and res.text == 'GenAtlas' and idv is not None:
sym = idv.text
if sym is None:
gn = poly.find(NS+'gene-name')
if gn is not None and gn.text: sym = gn.text
if sym: syms.append(sym)
target_symbols[pid] = syms
```

7.2 Obstacles and fixes (biological edges)

Obstacle	Why it happens	Fix
0 biological edges	matching UniProt/Entrez directly	Use the symbol bridge via hetionet tsv (7.1)
Drug has no targets	biologics/enzymes (heparin, streptokinase) have no DB	Leave it DI-only; it is the same as 359 existing sparse drugs. Not an e
Symbol not in Hetionet	gene absent from Hetionet's ~20k genes	Skip that one edge; others still connect the drug
Wrong relation type	drug-gene is not always 'binds'	CbG (6) is a safe default; refine with CuG/CdG if you have direction d

8. Step 6 - Clear Caches and Retrain

KnowDDI precomputes per-pair subgraphs and may cache them (files containing 'hop', or .pkl/.pt subgraph caches under data/drugbank/). If present, delete them so the new nodes and edges are picked up. Then retrain.

```
# remove any precomputed subgraph / hop caches if they exist
find data/drugbank -iname '*hop*' -o -iname '*.pkl' -delete
# retrain (uses the published KnowDDI config)
python pytorch/train.py --dataset drugbank
```

Note: There is no pre-trained checkpoint shipped with the repo. You train from scratch once; the resulting model knows all original drugs plus every drug you appended. New drugs are inert until this training run.

9. Validation Checklist (run before training)

- Every new ID is unique and > the previous global max (no collision with genes).
- Every endpoint in appended train.txt / BKG_file.txt lines exists as a node.
- No new drug is fully isolated: it has at least one DDI edge OR one biological edge OR a path through the KG.
- All appended type_ids are in 0-85; all KG rel_ids are valid relation integers.
- Unmatched DDI descriptions were logged, not silently assigned.
- Backups (*.bak) exist for node2id.json, train.txt, BKG_file.txt.

```
# quick sanity checks
import json
n = json.load(open('raw_data/Drugbank/node2id.json'))
ids = set(n.values()) |
set(json.load(open('raw_data/Drugbank/BKG_entity2Id.json')).values())
for ln in open('data/drugbank/train.txt'):
h,t,r = map(int, ln.split())
assert h in ids and t in ids, f"dangling DDI edge: {ln}"
assert 0 <= r <= 85, f"bad type id: {ln}"
print('train.txt OK')
```

10. Scaling Beyond NLEM (Future Drugs)

The same procedure adds **any** legally sold Nepali drug, not just the NLEM list. To industrialise it:

- Keep a single growing **alias table** (Nepali/brand/spelling to DrugBank ID) so manual matches are reused, never redone.
- Run the whole pipeline as one script that takes a plain list of drug names and outputs the three appended files plus a report (resolved / new / out-of-scope / unmatched).
- Always dry-run first (print counts, write nothing); only apply after the numbers look right.
- Capacity: the embedding table is fixed at 35,000 nodes; current max is 34,123, leaving ~876 free slots. Adding more than that requires raising num_nodes in pytorch/train.py before training.

Caution: If you ever add more than ~876 drugs in total, increase the hard-coded node table size (num_nodes = 35000) in pytorch/train.py, or the new IDs will exceed the embedding matrix and training will crash.

This guide documents the dataset-expansion procedure for MediSafe Nepal. The accompanying script append_drugs.py implements steps 1-6 automatically; this document is the reference for understanding, auditing, and extending it.